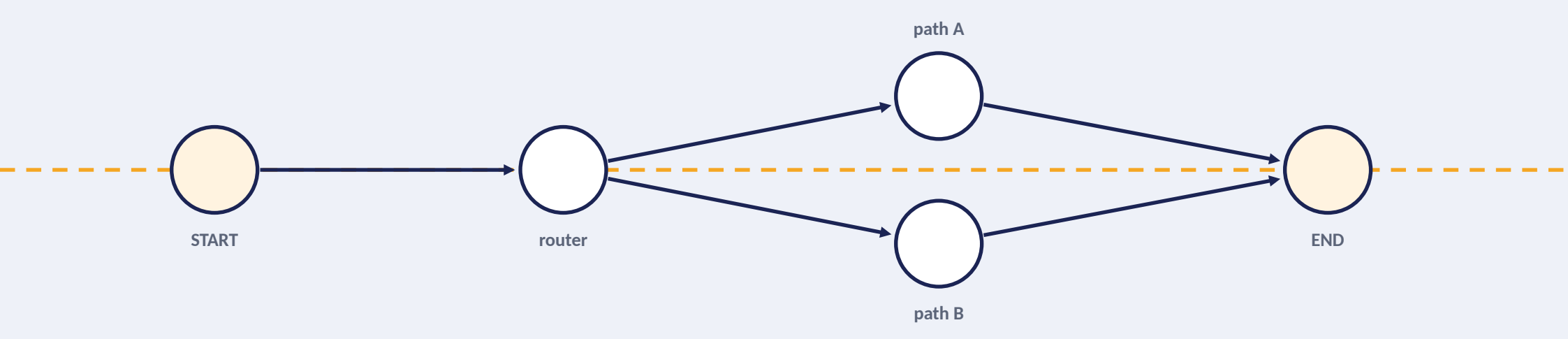
LANGGRAPH · CHAPTER 1 · FLASHCARDS

Why LangGraph?

Flow you can see and change

7 code cards on graphs, state, nodes and edges



Work and flow in one function

Plain Python mixes what each step does with how the steps connect

- Work lines classify, write and critique; flow lines skip, loop and count tries.
- To add a branch you must reread the whole function: a graph keeps the flow apart.

PYTHON

```python
def handle(email):
    kind = classify(email)          # work
    if kind == "newsletter":        # flow: skip this one
        return
    if "meeting" in email:          # flow: only sometimes
        busy = check_calendar()     # work: a tool call
    draft = write_reply(email)      # work (busy is never used: easy to miss)
    tries = 0
    while tries < 3:                # flow: loop with a counter
        score = critique(draft)     # work
        if score > 0.7:             # flow: good enough, stop
            break
        draft = revise(draft)       # work
        tries += 1
    send(draft)                     # work

# A sketch from §1.1, not code to run
```

Who decides what runs next?

In a workflow you draw every route; in an agent the model picks the route at runtime

- Start with a workflow: it is cheaper, predictable and easier to debug.
- Chain, workflow and agent are all graphs: no forks, forks you wired, a loop the model drives.

TEXT

```
chain     START -> retrieve -> prompt -> model -> END
          same steps, same order, every run

workflow  START -> classify --newsletter--> ignore -> END
                            --fyi---------> notify -> END
                            --other-------> reply  -> END
          you wired every branch in advance

agent     START -> model <------> tools
                      |
                     +--done--> END
          the model loops through its tools and decides when to stop
```

The state is a contract

A TypedDict declares every field that must survive from one step to the next

- Every node reads from and writes to these fields: no globals, nothing hidden in arguments.
- An explicit state is what later lets LangGraph save it, pause on it and merge parallel writes.

PYTHON

```python
from typing_extensions import TypedDict


class State(TypedDict):
    question: str    # what the user asked
    draft: str       # the current reply
    score: float     # how good the draft is

# One run's state, as a plain dict:
# {"question": "pricing", "draft": "Thanks!", "score": 0.0}
```

State in, partial update out

A node reads the full state and returns a dict of only the fields it changed

- Each returned key names a state field; LangGraph merges it in before the next node runs.
- Fields the node leaves out keep their values, so a node cannot clobber what others own.

PYTHON

```python
def score_draft(state: State) -> dict:
    points = 0.8 if "thanks" in state["draft"].lower() else 0.4
    return {"score": points}      # only the field this node changed

# The merge, traced:
#   before:  {"question": "pricing", "draft": "Thanks!", "score": 0.0}
#   returns: {"score": 0.8}
#   after:   {"question": "pricing", "draft": "Thanks!", "score": 0.8}
```

Fixed edges and forks

add_edge always goes to one node; a conditional edge asks a router function where to go

- The router reads the state and returns the next node's name, or END to finish.
- An edge may point backward: here "score" can send the run back to "draft", a loop.

PYTHON

```python
builder.add_edge(START, "draft")     # normal edge: always
builder.add_edge("draft", "score")   # after draft, always score


def route_after_score(state: State) -> str:   # the fork's decision
    return END if state["score"] >= 0.7 else "draft"


builder.add_conditional_edges(
    "score",               # after this node runs...
    route_after_score,     # ...call the router with the state...
    ["draft", END],        # ...which may choose only these
)
```

LISTING 1.1 · COMPILE AND RUN

Wire it, compile it, invoke it

StateGraph, add_node, add_edge and compile() turn plain functions into a runnable graph

- The input holds only question: the nodes fill in draft and score along the way.
- The nodes never mention each other; their order lives entirely in the wiring.

PYTHON

```python
def draft_reply(state: State) -> dict:
    return {"draft": f"Thanks for asking about {state['question']}!"}


builder = StateGraph(State)                    # 1. choose the baton
builder.add_node("draft", draft_reply)         # 2. name -> function
builder.add_node("score", score_draft)
builder.add_edge(START, "draft")               # 3. lay the track
builder.add_edge("draft", "score")
builder.add_conditional_edges("score", route_after_score, ["draft", END])

graph = builder.compile()                      # 4. check the wiring
print(graph.invoke({"question": "pricing"}))
# {'question': 'pricing',
#  'draft': 'Thanks for asking about pricing!', 'score': 0.8}
# 0.8 >= 0.7, so the router sends the run to END on the first lap
```

Two beginner mistakes

Editing the incoming state in place is lost silently; a graph with no way in won't compile

- Only the returned dict is merged: an in-place edit with an empty return changes nothing.
- compile() checks the wiring: every edge must hit a real node and START must lead somewhere.

PYTHON

```python
def bad_score(state: State) -> dict:
    state["score"] = 0.8      # edits the incoming state in place...
    return {}                 # ...and returns no update

# A graph whose only node is bad_score:
app.invoke({"question": "pricing", "draft": "Thanks!", "score": 0.0})
# {'question': 'pricing', 'draft': 'Thanks!', 'score': 0.0}  <- edit lost

builder = StateGraph(State)
builder.add_node("draft", draft_reply)
builder.add_edge("draft", END)          # no edge from START
builder.compile()
# ValueError: Graph must have an entrypoint: add at least one edge
#   from START to another node

# Checked on langgraph 1.2.6
```